AI 100 Final Project — Reflection Report

Kreesh Patel | Spring 2026 | LLM used: ChatGPT

Overview

For this final project, I took the MNIST CNN I built for the midterm and added 10 different bugs to it on purpose. For each bug, I wrote down what I thought was wrong before asking the AI for help. Then I used the prompt from the assignment to ask ChatGPT, got a 'Bad' rating with some hints, and wrote a better explanation. The spreadsheet (*final_project_bad_cases.xlsx*) has all 10 cases. This report covers what I learned: first about working with an AI as a study partner, then about building AI systems in general.

I. What I Learned About GenAI

The hints made me think instead of just giving me the answer.

When I gave the AI a weak guess, it did not just tell me the fix — it asked me questions that made me think harder. For Case 5 (I forgot *optimizer.zero_grad()*), my first guess was 'maybe the learning rate or the model is wrong.' The AI did not say 'you forgot zero_grad.' It asked:

> *"What does optimizer.zero_grad() do? What happens to .grad if you never call it? After many batches, how big do the gradients get if they keep adding up? How would Adam react to that?"*

Those questions pointed me at the right area without giving away the answer. I had to do the actual thinking myself, which is different from how I usually use AI — I usually paste the error and copy whatever fix comes back. Here I had to think.

The error message often had the answer already.

Case 1 was the clearest example. I changed *nn.Linear(64*7*7, 128)* to *nn.Linear(64*14*14, 128)* and got this:

```
RuntimeError: mat1 and mat2 shapes cannot be multiplied (256x3136 and
12544x128)
```

My first reflection said 'something is wrong with the linear layer.' The AI asked, 'Where do the numbers 3136 and 12544 come from?' That made me trace the image size: 28 → 14 after the first pool, then 14 → 7 after the second pool. So 64*7*7 = 3136 (right) and 64*14*14 = 12544 (wrong). The error message basically told me the answer, but I had not read it carefully. Cases 2 and 3 had the same pattern. The AI's value here was not knowing more than me — it was making me read what was already there.

The AI can only work with what I tell it.

For Case 9, I swapped the order of *optimizer.step()* and *loss.backward()*. If I had told the AI 'training is slow,' it could have suggested longer training or a different optimizer, neither of which

would help. But because I said I 'rearranged a couple of lines,' the AI knew to ask about what step() needs to work. Bad input gives bad advice, even with a good AI.

The AI helps me think but does not give me the answer.

The Socratic prompt only asks questions and then stops. So I could not just copy a fix. After Case 6, I had to figure out on my own that on a [batch, classes] tensor, *dim=0* reduces over the batch and *dim=1* reduces over the classes. The AI helped me see the right question, but I had to do the work to answer it.

Writing a bad first reflection on purpose was useful.

For some cases I let myself write the lazy guess (Case 6: 'maybe my model is overfitting'; Case 8: 'the model must be broken'). Watching the AI push back on those weak guesses showed me what bad reasoning looks like. I would never normally type 'maybe the data is corrupted' to ChatGPT, but doing it on purpose made me see how much better the second reflection was.

II. What I Learned About Building an AI System

Layer shapes have to match.

Cases 1, 3, and 4 all came down to one part of the model expecting one shape and the next part expecting a different shape. Case 1 was the flattened conv output vs. the linear input. Case 3 was the input channel count (1 for grayscale) vs. the conv's expected channels. Case 4 was a PIL image going into Normalize, which needs a tensor. Building an AI system means making sure shapes line up at every step.

The order of steps in the training loop matters a lot.

Cases 5 and 9 only changed the order or removed one line. Case 5 dropped *optimizer.zero_grad()*. Case 9 swapped *step()* and *backward()*. The functions still ran, but training failed because each step needs the right state from the step before it. The right order is: zero gradients → forward pass → loss → backward → step. Anything else breaks.

Silent bugs are worse than crashes.

Cases 5, 6, and 10 do not error out. They print numbers and finish. But Case 6 only got 10% test accuracy from a model that was actually fine, Case 10 dropped a few percentage points, and Case 5 trained badly. If I did not know my baseline was 98.84%, I could have shipped any of these and not noticed. Code that runs without errors is not the same as code that is correct — I need to compare to a baseline to know.

The model and the loss function have to match.

Case 2 (output size 9 instead of 10) and Case 7 (NLLLoss with raw scores instead of log-probabilities) are both 'model and loss do not agree' bugs. The loss function decides what the model's last layer needs to output. They go together, so I cannot pick one without thinking about the other.

A wrong number can break things just as badly as a wrong line of code.

Case 8 changed lr from 0.001 to 10.0. No code is wrong by itself, but the system gives NaN. Hyperparameters can break a model just like a typo can. They count as part of the system, and they need to be tracked and version-controlled too.

Shuffled data matters.

Case 10 used *shuffle=False* on the training loader. Nothing crashes, but accuracy drops a few points. The optimizer assumes each batch is roughly random; if the data is always in the same order, training is slower and less stable. The bug is not in the code — it is in what data the code sees.

Read the error message before guessing.

I noticed that for the crash cases (1, 2, 3), my first guess always tried to find a fix before I really read the error. PyTorch errors usually tell you the mismatch directly — *(256x3136 and*

12544x128) is the answer if you read it. Slowing down to read the error before guessing was the biggest habit I changed during this project.

Closing thought

What surprised me most is that I learned more from the bugs I added on purpose than from the working model I built first. The midterm taught me how PyTorch *works*; this project taught me how it can *break*. Seeing both helped me know what I really understood vs. what I was just copying. The Socratic prompt was uncomfortable when I read the 'Bad' verdict, but that was what made me write a better second reflection.